



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Készítette:	Király Levente
Neptun-kód:	CNDFT2
Szakirány:	Információs Rendszerek
E-mail cím:	kirlevente04@gmail.com
Konzulens(ek):	Pálffy Miklós Gábor
E-mail címe(ik):	palfy.miklos@vik.bme.hu

**Téma címe: Kamerás pékáru polctöltöttség-felismerő
rendszer tervezése és Proof Of Concept (POC)
megvalósítása**

Feladat

Megtervezni és megvalósítani egy gépi tanulási modellt, valamint implementálni mellé egy segédprogramot. A program képes feldolgozni egy előre elkészített képet/videót, amely pékáruval teli tálcákat tartalmaz. A modell segítségével képes meghatározni, hogy az adott képen/videón egyes tálcák töltöttségi szintje a skála melyik részébe tartozik (kicsi/közepes/magas).

2025/2026. 2. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1 Bevezető

A modern korban számos helyen előkerül a gépi látás megvalósítása, mind a közlekedésben, egészségügyben, valamint sok más helyen is. Például egy autópályán utazó jármű detektálható, meghatározható annak helye, sebessége, sőt, ezt az autóbba implementálva még önműködésre is bírható. Röviden fogalmazva, mára már nagy mértékben befolyásolja a gépi tanulás, azon belül is a gépi látás a mindennapi életet. Ezt alapul véve szerettem volna egy olyan projektet választani, amely ezeket felhasználva segíti az emberek munkáját a hétköznapiakban.

Az üzletekben nem ritka, hogy az árucikkeket valamilyen módon ellenőrzik, ezáltal tudják azt, hogy mikor kell polcfeltöltést végezni. Eddig a szaküzletek előszeretettel használtak RFID (Radio Frequency Identification) -t ennek megvalósítása, de még mindig széles körben elterjedt a súlyalapú vizsgálat, illetve a dolgozók által végzett manuális számolás is. Ugyanakkor ezek megvalósítása költséges lehet, nagy lehet a hibaarány, hogyha kisméretű tárgyak megszámlálásáról van szó. Ráadásul az RFID technológia bevezetése nemcsak drága, de chipekkel ellátott árcímkéket is megkövetel, ami nem minden esetben kivitelezhető.

A pékáruk polcainak feltöltéskezelése ennek tekintetében viszonylag nehéz dolognak minősül. Az RFID nem megvalósítható, a folyamatos vizsgálat kivitelezése pedig macerás.

Ezzel szemben egy kamera felszerelése kivitelezhető, valamint egy gépi modell betanítása költséghatékony, nagymértékben skálázható, illetve emberi erőforrásokat szabadít fel. Ez nagyban megkönnyíti a dolgozók munkáját, valamint a működtető cég számára is nagyobb nyereséget hoz.

Ennek fényében valósítottam meg mind a modell tanítását, mind a segédsoftvert, amely képes egy képet feldolgozni, majd felhasználói útmutatásra dobozt létrehozni a megadott tálcára, amely a pékárut tartalmazza. A tálcáról képes eldönteni a töltöttségi szintet, így azt nem kell külön vizsgálni, elég csak újból feltölteni, ha alacsony szintet mutat.

1.2 Elméleti összefoglaló

A probléma megoldásához a gépi látás eszközeit alkalmaztam. Ezek az általános gépi tanuláson túlmutatnak. A gépi tanulást csak röviden szeretném összefoglalni. A modellnek elengedhetetlen feltétele, hogy megfelelő, jó minőségű tanító adatot kapjon. Később az adatokat tanító, teszt és validációs adatahalmazra kell bontani, majd különböző algoritmusok – például DNN (Deep Neural Network) – iteratív futtatásával és visszacsatolásával lehet tesztelni, hogy mennyire volt jó a tanítás mértéke.

A gépi látás a gépi tanulás egy speciális, képi adatokra fókuszáló részterülete. A modell adatként egy képet kap, majd azt próbálja megérteni, hogy egyes képek hogyan térnek el egymástól, illetve mi a kép fő jellemzője. Ehhez egy numerikus reprezentációt is felépít a látottakból, ebben raktározza az említett tulajdonságokat. A modell CNN (konvolúciós neurális hálózatok) segítségével tudja feldolgozni a képet. Ezen belül használ konvolúciós rétegeket, illetve szűrőket is. Részletesebb leíráshoz ajánlom elolvasni az NVIDIA által kiadott Computer Vision[1] dokumentációt is. A képi tulajdonságok és az azok felfedezésére szolgáló eszközök megértése is nagyon fontos. Az annotáció – amely a tanító adatokon a végzett felcímkézést jelenti – során el kell dönteni, hogy milyen módon kell a képeket dobozolni. Így nem árt tudni az eszközök részletes működését, mert különben probléma léphet fel.

Fontos megjegyezni, hogy a gépi látás során különféle képfeldolgozási feladatokról van szó, és az egyes szakmai kifejezések között jelentős eltérések vannak. Ezen szavak ismerete szinte

elengedhetetlen. Az osztályozás (Classification) eldönti, hogy a kapott képen az előre megadott osztályok közül mi szerepel, a kép egészét nézi és csak egy választ ad. A detektálás (Object Detection) egy objektumról képes meghatározni a helyzetét is, azaz egy képen több jelölést is ad. Mindegyik objektumot külön osztályozza, majd egy jelölő téglalapot rajzol köréjük. A szegmentálás (Segmentation) a detektálásnál is precízebb vizsgálat, a téglalapos jelölés helyett a detektált alak egészét körvonalazza. Érdekes még megjegyezni a felismerés (Recognition) fogalmát is. Ez az osztályozással ellentétben nem csak felismeri az objektumot, de képes ahhoz egy címkét létrehozni és tárolni, így azt nyomon lehet követni. Az első három kifejezésre a korábban említett NVIDIA Computer Vision[1] olvasmányban lehet példát találni.

A videós kielemezés egyik fő problémája, hogy a kiértékelés időbe telhet, így el kell dönteni, hogy a modell tulajdonságai közül melyiket vetjük el. Egyik ilyen kompromisszum a gyorsaság és minőség között jön létre. Tehát ha a modellünk gyorsan végzi el az osztályozást, akkor előfordul, hogy sokat hibázik, illetve vice-versa. Az iparban bevett szokás, hogy valamilyen előre megírt architektúra használandó, az én esetemben a YOLO (You Only Look Once) -ra esett a választás. A YOLO működési elve, hogy a kiértékelést egyetlen lépésben végzi el. Ez jelentősen megnöveli a gyorsaságot, így a valós idejű feldolgozásra kifejezetten ajánlott. A hálózat kimenetként megadja a felismert objektumok koordinátáit, a hozzájuk tartozó osztályt, valamint egy confidence score-t, amely megmutatja, hogy mennyire volt biztos az adott predikcióban. A működési elvét nagyon részletesen leírja az Encord által publikált blog[2].

A projekt során – illetve a való életben is – előkerült olyan probléma, miszerint a program nem jó helyen keresi az adott objektumokat, valamint az egész képet nézi, miközben elég lenne csak a felét vizsgálnia. Erre találtam megoldást a ROI (Region Of Interest) implementálásakor. A ROI pont ezeket a problémákat küszöböli ki, megadható egy adott hely a képen, amit vizsgálni kell. Ez a gyorsaságot is növelheti, valamint elkerülhető vele az a kellemetlen eset, amikor a modell felismer olyan objektumokat, amelyek nem oda tartoznak.

Ezekkel az ismeretekkel már fel lehet készülni a második fejezetben elhangzott lépésekre. Ugyanakkor a konkrét megvalósítás során akadtak új megközelítések is, amelyeket majd az adott fejezetben tárgyalok ki.

1.3 A munka állapota, készültségi foka a félév elején

A kiadott témával ezelőtt nem foglalkoztam. Korábbi tárgyakon (Adatelemzés Mélytanulási Módszerrel) a gépi látás előkerült, ugyanakkor nem ilyen mélységben.

A feladatot önállóan oldottam meg, ugyanakkor a program ellenőrzése során a konzulensem, Pálffy Miklós Gábor is szolgáltattott képeket pékárukat tartalmazó tálcákról.

2. Az elvégzett munka és eredmények ismertetése

2.1.1 Feladat megtervezése

Legelső feladatként megterveztem a modell implementálásához szükséges feltételeket, ezek mellett egy alsó becslést, hogy mennyi tanító adatra lesz szükség a modell felépítéséhez. A tervezés során olyan gyakorlati kérdéseket is tisztázni kellett, mint a tanítóadatok feldolgozásának módja, vagy hogy milyen elvárt metrikákat (például megbízhatósági küszöb, valós idejű futás képessége) kell teljesítenie a kész modellnek.

2.1.2 Nyers adatok gyűjtése

A feladat első részeként – mint azt korábban említettem – a modell betanításához szükség van bemeneti adatokra, amelyek péksüteményeket reprezentálnak. Az adatgyűjtésnek két fajtája van, egyik az, hogy egy open-source, ingyenesen elérhető adatbázisból szerezem meg a képeket, míg a másik a saját kézzel való gyártás. Én a második, manuális képgyűjtést választottam, annak érdekében, hogy minél több új dolgot kipróbálhassak a projektmunka során.

Az adatgyűjtés során több áruházba is ellátogattam, figyelve arra, hogy minél több variancia legyen egyes üzletek között. Ezen boltoknál eltérő volt – a teljesség igénye nélkül – a pékárukat tartalmazó tálca színe, egy tálcán milyen mennyiségben vannak az áruk, egyes polcok dőlésszöge, tálcák fényvisszaverődése (egyes tálcák előtt található volt egy műanyag ajtó/fal, amelyek visszaverte a fényt).

A boltokban videófelvételt készítettem a polcokról, balról jobbra haladva, figyelve arra, hogy több szögből is felvegyem azokat. Fontos szempont volt, hogy a tálcákról kiegyenlített mértékben készítsek képeket, tehát egyforma arányban szerepeljenek az alacsony, közepes és magas töltöttségű tálcák is. Erre egy nagyszerű példa az **1. kép**, ahol mindhárom szintből 3 darab tálca van feltüntetve. Ez kifejezetten nehéz feladatnak bizonyult, ugyanis egyes üzletekben hihetetlenül alacsony volt a kínálat. Ez azért volt problémás, hiszen a tálca színe a többitől eltért, így nem volt reprezentálva elégszer azon tálca színe magas töltöttségnél. Ezt csak úgy tudtam orvosolni, hogy más, hasonló márkájú boltban is készítettem videót.



1. kép. A péksüteményeket tartalmazó tálcák egyik fotója

Miután elkészítettem a videókat – összesen 15 darab, 10-20 másodperces videó keletkezett a projekt elkészüléséig – azokat valamilyen módon átkellett alakítanom bemeneti képekké. A számomra legegyszerűbb megoldás egy Python script írása volt, amely a bemeneti videókat feldolgozta és egy paraméterként megadott időközönként képet készített a videóról. Mivel a videók 60 FPS (Frame Per Second) -sel lettek rögzítve, paraméterként 30-at adtam meg. Ez a későbbiekben túl sűrűnek bizonyult és sok lett az egyforma kép, így később a 60-as paraméterszám mellett döntöttem. Végeredményben a manuális szortírozás után – ilyen volt a duplikátumok és a homályos képek kiszűrése - 122 darab bemenő képet hoztam létre.

2.1.3 Bemenő adatok annotálása

A modellek tanítása során nem elegendő csupán az elkészült képekről egy mappát beadni. A tervezés során a YOLO architektúrák közül a YOLOv8 modellt választottam. Ennek egyik előnye, hogy nagyon stabil, ugyanakkor – mint más egyéb modellek esetén is – eltérő mappa hierarchiába kell megadni az adatokat. Egy nagyon jó összefoglaló a Labelformat [3] oldalán található, ahol minden egyes modellhez leírja, milyen struktúra kötelező. Egyszerű megfogalmazással, a képek mellé egy címkét (label) is meg kell adni, amely azt mutatja meg, hogy a keresett objektum koordinátája hol van.

A manuális címkézés – más szóval annotálás – gyakran elterjedt szokás. Nem véletlen, hogy számos segédprogramot létrehoztak, ennek megkönnyítésére. Egyes programokban nem ritka, hogy a címkézést már nem is kézzel végzik, hanem (AI) Artificial Intelligence asszisztált módon. Én ezt a funkciót nem próbáltam ki sajnos, nem éreztem szükségnek.

Az annotáló programok közül nekem a Roboflow-ra esett a választásom. A program előnye, a többihez képest, hogy képes böngészőből futni, ingyenes, illetve modell kiválasztása után automatikusan hozza létre a megfelelő mappa felépítést.

Az annotálás során több gondolkodásmenet is felmerült, amely a tervezés során nem fogalmazódott meg bennem. Egyik ilyen gondolatmenet az volt, hogy a péksütemények címkézése helyett magát a tálcát kell címkézni, hiszen ennek elmulasztása során az üres tálcákat nagy eséllyel nem fogja megtalálni a program – még ha az üres tálcákat be is jelöltem, az egyetlen „low” címke miatt nem tudja megtanulni, hogy mind az üresekre, mind a majdnem üresekre is ugyan olyan címkét tegyen. Eredményül két annotált adatbázisom lett, ahol ezeket külön választottam. Az annotálás menetét a **2. kép** mutatja be, ahol a bal oldal az első adatbázisból, jobb oldal a második adatbázisból való egy darab címkézést mutatja be.

Fontos megjegyezni azt is, hogy második próbálkozásra már nem csak egyszerű téglalapokat használtam, hanem a tálca éleit követve precízebben mutattam meg a modellnek, hogy mit tanuljon meg.

A Roboflow szintén nagy előnye, hogy képes előfeldolgozással is segíteni a tanítást, így az nem tart annyi időbe. A képméreteket 512x512 pixeles méretre torzította és orientálta.

Végül a kész képeket augmentáltam is, amely azt jelentette, hogy az eredeti képeket valamilyen módon úgy eltorzítottam, hogy azt a modell jobban megértse. Külön kiemelem a fényerősség, illetve a homályosítás eszközét. Ezek arra szolgálnak, hogy különböző környezeti változás esetén – például lekapcsolják a tálcákat megvilágító fényforrást - is érzékelhető legyen az objektumok ott léte. A homályosítás annak érdekében van jelen, hogy minél szélesebb körben lehessen használni a modellt, azaz egy kisebb felbontású kamera esetén is.



2. kép. Az annotálások különbsége, ahol a bal oldal az első, jobb oldal a második verziót mutatja be

Végezetül, a tervezés során tett feltétel – miszerint az osztályok eloszlása egyenlő kell, hogy legyen - is megvalósult. Az eloszlást az *1. táblázat* mutatja meg, amely a már annotált képeket is tartalmazza. Összesen – a második adatbázist számolva – 300 képet tartalmaz a második tanító állomány. Ugyanakkor mindkét állománnyal végeztem tanítást, annak az eredményét és a változás okait a betanítás menete után fejtem ki részletesen.

A képen található osztályok nevei, angol kifejezéssel	Az adott osztály darabszáma
High	174
Medium	169
Low	108

1. táblázat. Az osztályok összes előfordulása a felcímkézett képeken

2.1.4 Modell betanítása

A modell betanítása a képek annotálása után egyszerű feladat volt. A Roboflow megfelelő beállításával automatikusan létrehozta az adatszerkezetet. Ezt egy Python Notebookban betöltöttem – ez is egyszerű volt, hiszen a program megadja a beszúrandó kódrészletet, valamint egy rejtett token, amely projektenként eltérő-, majd egy egyszerű YOLO modellt megfelelő paraméterezéssel betanítottam. A képek csoportosítását még a segédprogramban elvégeztem, így azt már nem kellett a forráskódban szétszednem tanító, teszt, illetve validációs adataira.

A tanítás során – annak ellenére, hogy egyszerű feladatnak bizonyult – akadtak bonyodalmak. Egyik javítandó probléma volt az adatszivárgás. A Roboflow feljánl az elkészítés előtt egy olyan funkciót, amellyel automatikusan szétosztja tanító és teszt adathalmazokra a képeket, amelyet első alkalommal bejelöltem. Ugyanakkor ez olyan esetben hasznos, amikor a képek teljesen eltérnek egymástól. Mivel nekem egy videóból lettek kivágva képkockák, könnyen előfordult, hogy egyes elemek ismétlődtek, még az adatduplikátumok kiszűrése esetén is. Ezáltal a validáció teljesen értelmét veszítette, hiszen már láthatta előtte az adott objektumot, így tanulás nélkül is megtalálta azt. A problémát úgy oldottam meg, hogy különböző üzletek különböző funkcionalitást végeztek el, tehát nem kerülhetett olyan üzlet a tesztelésbe (azonos tálca szín, dőlésszög) amely már a tanítás során szerepelt.

Összesen 5 darab tanítást végeztem el. A modellek a .pt kiterjesztésű fájlok. A modellek tanítása során statisztikai adatok is létrejöttek, ilyenek például a konfidencia görbék és a konfúziós mátrixok. A minimum konfidenciaszintet ezek alapján 0,32-nek választottam, ez lett az alapértelmezett érték. Ezesetben, ha eltérés keletkezik, azt az adott részben fejtem ki.

Az általam használt modell a legutolsó tanítás volt, amely a második adatbázis annotálásából származik. A tanítás során nem volt jelen adatszivárgás, illetve itt egy egész tálcát csak egy tálcának néztem, hogyha az tele volt. Erre a későbbiekben best5.pt modellként fogok hivatkozni. A másik szóra érdemes modell az első képsorozatból – amelynél tálca nélkül címkéztem a péksüteményeket – tanított modell. Ezt is megtartottam, mivel érdekelt, hogy hogyan működik azzal a modellel a kiértékelés. Ez a modell a best2.pt nevet kapta.

Ettől a ponttól, az alapértelmezett modell név a best5.pt, tehát ha egy elkészített YOLO modellre hivatkozok, de azt nem nevezem nevén, akkor erre értendő az állítás. Hogyha ettől a modelltől eltérek, akkor külön feltüntetem a neveket a leírás során.

2.1.5 Videók és képek kiértékelése

A tervezés során a képek kétféle feldolgozására jött elő ötlet. Egyrészt érdemes olyan segédprogramot írni, amely egy videót kér be, majd egy másikat is, amely csak képeket. Ez azt a célt szolgálja, hogy a videófeldolgozó megmutatja mennyire hatásos a detektáció, akár a követés. A képfeldolgozó rendszer pedig a helyességet figyeli. Mivel a gyakorlatban egy fixált kamera nézné a tálcákat, így több képkocka jelentősen hasonlít egymásra, a probléma jobban tükrözi a képfeldolgozós részt. Később ezt a gondolatmenetet lehet bővíteni is.

A segédprogramot a cv2 könyvtár segítségével valósítottam meg. A program a kapott videót képkockaként vizsgálja, majd mindegyiket YOLO modell szerint kiértékeli. A detektált objektumok köré dobozokat rajzol, illetve az objektum osztályát és konfidencia egységét is leírja. Több olyan videón is leteszteltem, amelyet a tanítás során - mind tanító, mind tesztelő adathalmazon értendő – még nem látott. Egyik ilyen kiértékelt képkocka a **3. képen** látható.



3. kép. A kiértékelt videó képkockája, amely a dobozolást mutatja be.

A dobozok színei jól láthatóan osztályonként változnak. A harmadik sorban a tálcák elkülönítése megvalósult, a negyedikben azonban már nem.

2.1.6 Kiértékelés más módszerekkel

A videók kiértékelése szemmel láthatóan jól – csupán a POC megvalósítás szempontjából – működött, ugyanakkor a képek feldolgozása során új probléma merült fel. Egyes esetekben a kép túl távolról készült és az egyszerű YOLO modell nem volt képes ránézésből felismerni a meglévő objektumokat. Egyes esetekben, amelyet a **4. kép** illusztrál egyetlen tálcát sem ismert fel. A probléma megértése viszonylag egyszerű feladat, mivel a tálcák a kép túl kicsi részét képezik, azt a modell nem képes értelmezni, nem kezeli annak módján.

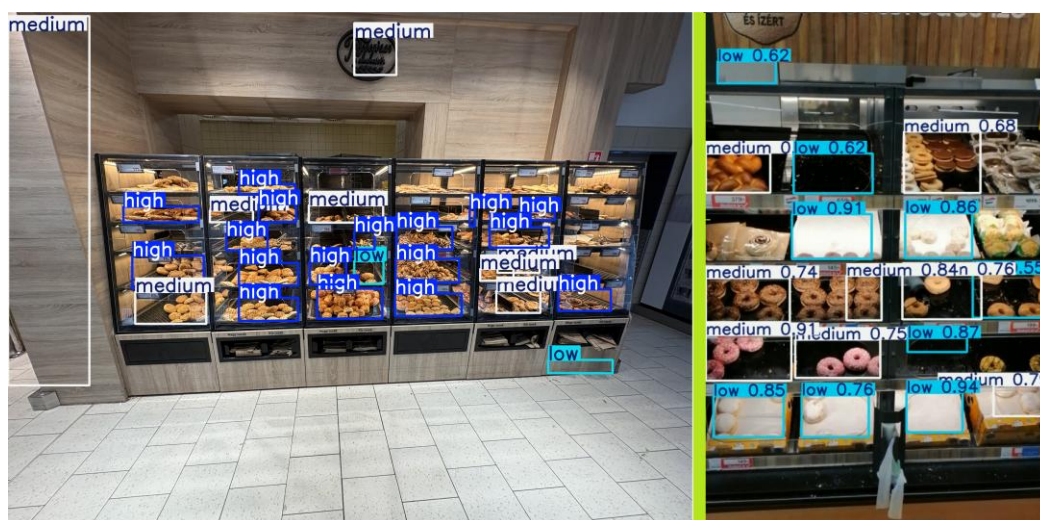


4. kép. Egy távolról készült kép a tálcákról, amelynél nem ismer fel a modell semmit

Szerencsére a probléma nem egyedi eset, számos gyakorlati példa van, ahol ez előjött már. Ezáltal van bevált módszer a kezelésére, amelyet már elkészítettek. A YOLOv8 esetében implementálható egy SAHI (Slicing Aided Hyper Inference) szeletelő, amely segítségével képes megtalálni kisebb objektumokat a képen, nagy felbontásban is. A működése egy csúszóablakon alapul, ahol a modell nem a kép egészét vizsgálja, hanem a megadott ablak dobozát. Ennél természetesen bonyolultabb a folyamat, amelyet Eran Feit blogja[4] mutat be részletesebben.

Ez a módszer sokkal pontosabban megvizsgálja a képet, ugyanakkor jelentősen lassabb lesz a feldolgozás. Általánosan elmondható, hogy egyenesen arányosan növekszik a feldolgozási idő a szűrő méretével. Ez jelentősen lecsökkenti a kiértékelés gyorsaságát.

A megvalósított SAHI és YOLO kiértékelést az előző **3. kép** videóján, illetve a **4. képen** teszteltem le. Az **5. kép** mutatja be a kiértékelési dobozokat.



5. kép. Az előző képek kiértékelése SAHI módszerrel

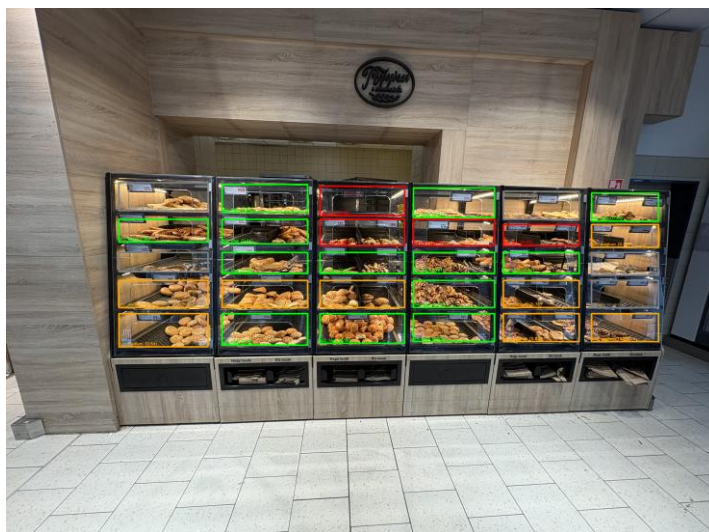
Jól látható, hogy most már a kép teljes részén jelennek meg dobozok, nagyrészt eltalálja azoknak a helyét. Ugyanakkor az is észrevehető, hogy még így sem teljes a detektáló képesség, illetve sok a hamis pozitív (false positive), tehát olyan részeket is detektál tálcaként, amelyek nem a keresett részbe tartoznak.

Fontos megjegyezni, hogy itt mindkét említett modellel teszteltem. Összességében elmondható volt az, hogy a best2.pt modell a kis mennyiségű tálcákat – hacsak nem teljesen üres – szinte mindig tudta detektálni. Ez a modell sokszor jobb eredményeket mutatott a best5.pt modellel szemben, annak rovására, hogy az üres tálcákat egyáltalán nem találta meg.

2.1.7 Detektálás elhagyása

Az előző feltevés – miszerint hamis pozitív detektálások is jelen vannak – problémájára is találtam megoldást. A lokalizálást nem a modellre bízom, hanem kézzel adok meg a képen doboz koordinátákat – úgynevezett ROI módszer megvalósításával - amelyeket ki kell, hogy értékeljen. Ezáltal elkerülhető az említett probléma, valamint a hatékonyság is megnő azzal, hogy elhagyom a SAHI megvalósítást. Azaz jelenleg egy képet adok be a modellnek, adott dobozokkal, ezeket a dobozokat értékeli ki a YOLO modell, osztályozást hajt végre, valamint megadja a konfidencia szintjét egyes predikciókhoz.

Ezzel a kiértékelési módszerrel már biztosítható, hogy csak a keresett tálcákat vizsgálja a modell, ezeknek méri a töltöttségi szintjét. Ennek egyik kimenete a **6. kép**, ahol a szürke doboz azt reprezentálja, hogy nem ismert fel adott konfidencia intervallum esetén osztályt.



6. kép. Kiértékelés, ahol a dobozok színei a töltöttséget reprezentálják.

Ezek után megvalósítottam egy felhasználói programot is, amellyel egy képet megnyitva a felhasználó dobozolni tud. A felhasználónak elég a dobozok sarkát megadnia, az egeret a kezdőponttól a végpontig nyomva kell tartani, majd felengedni. Külön probléma volt az alulról-felfele jelölő irány a kiírások tekintetében, amelyet orvosolni kellett.

A program minden egyes kijelölt dobozt egy zöld színnel körbe rajzol. Implementáltam azt a funkciót is, amely érzékeli, hogy mekkora a doboz mérete, így az nem fog felrajzolni dobozt véletlen gombnyomás esetén. A program bezárása után létrejön egy kiértékelt kép, amely megegyezik az előző, ROI és YOLO használati kombinációjával.

Az elkészített programra nagy szükség van. A rekeszek bővítése nagyon egyszerű feladat,

csak új dobozolást kell megvalósítani. Ezek mellett nagyon könnyen implementálható a program videók elemzésére is. Mivel a kamera állása fix, egy darab mintakép dobozolásával az egész videóra kiterjeszthető annak használata.

Jelen esetben is mindkét modellt használtam a tesztelés során, itt a best5.pt adott jobb eredményeket, a best2.pt sokszor tippelt valótlanul. Megfigyeltem, hogy a kiértékelés során, ha nem volt biztos egy modell az adott tálcában, akkor nagy valószínűséggel közepes (medium) eredményt adta predikcióként.

2.2 Összefoglalás

Az üzletekben lévő pékáruk töltöttségének vizsgálata a gyakorlatban komplex feladat. Apró méretük miatt nehéz a számolásuk, illetve fizikai módon sem létezik egyszerű megoldás. A félév során ennek a kamerás felismerő rendszernek a Proof Of Concept megvalósítása volt a célom. Három szintet terveztem felismerni, amelyek a tálcákat alacsony, közepes és magas kategóriába sorolták.

A feladat megoldása érdekében a YOLOv8 objektumdetektáló architektúrát alkalmaztam. Az ehhez szükséges tanító adatot saját kézzel generáltam. Különböző üzletekbe ellátogattam és videófelvételt készítettem. Később képekre vágtam ezeket.

A képeket egy Roboflow nevezetű segédsoftverrel annotáltam, majd betanítottam a modelletemet. Ebből többféle verzió is elkészült.

A feladat ideálisan nehéznek bizonyult, amely a videós tesztelésnél kiderült. Külön odafigyelést követelt, hogy eltérő boltokban, eltérő környezetekben, rossz látási viszonyok mellett is működjön a modell.

A problémákra különböző megoldásokat találtam. Azokon a képeken, amelyeken az objektumok kis mérettel jelennek meg, SAHI-t alkalmaztam. Emellett felhasználtam a ROI eszközt is, amely csak az előre megadott tálcákon fog értékelést végezni. A végső programban nem implementáltam SAHI eszközöket a lassú feldolgozás miatt.

A legfontosabb eredmény a végleges, optimalizált architektúra és a hozzá tartozó segédsoftver elkészítése volt. Ennek segítségével a felhasználó manuálisan meg tudja adni a tálcák dobozait. A program használatával egyszerűsödött a feladat, hiszen új tálcák esetén egyszerű a bővítés, rossz működés esetén pedig csak a modell cserélése szükséges, amely csupán egy paraméter átírásába kerül.

Következtetésképpen a POC rendszer megvalósítása sikerült. A jelenlegi modell pontossága bőven növelhető. Ezt leginkább több tanító adattal és jobb annotálási módszerekkel lehet elérni. Ezt a modellt a már meglévő architektúrába nagyon egyszerű implementálni.

Még több energiabefektetéssel egy valós idejű kamerás hálózattal összekötve a rendszer egy gyakorlatban is bevezethető ipari megoldássá fejleszthető.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

A tanulmányozott irodalom jegyzéke:

- [1] NVIDIA Corporation, Computer Vision Glossary, Elérhető: <https://www.nvidia.com/en-us/glossary/computer-vision/> Utolsó letöltés ideje: 2026. május 12.
- [2] Nikolaj Buhl, Encord, YOLO Object Detection Explained: Evolution, Algorithm, and Applications, Elérhető: <https://encord.com/blog/yolo-object-detection-guide/> Utolsó letöltés ideje: 2026. május 12.
- [3] Lightly, Labelformat, YOLOv8 Object Detection Format, Elérhető: <https://labelformat.com/formats/object-detection/yolov8/> Utolsó letöltés ideje: 2026. május 12.
- [4] Eran Feit, YOLOv8 Small Object Detection with SAHI in Python (Sliced Inference), Elérhető: <https://eranfeit.net/how-to-detect-small-objects-with-yolov8-and-sahi/> Utolsó letöltés ideje: 2026. május 12.